VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

ABHINAV CHANDRASEKAR

(1BM24CS007)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by ABHINAV CHANDRASEKAR (1BM24CS007), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026.

The Lab report has been approved as it satisfies the academic requirements in respect of OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program1a

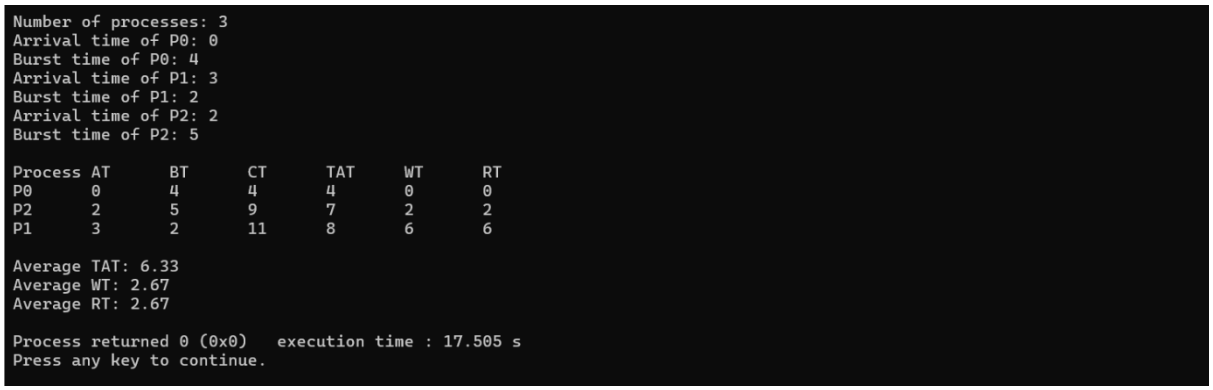
Question: Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time for FCFS and SJF (SJF+SRTF)

Solution for FCFS –

```
#include <stdio.h>

struct processes
{
   int ID, AT, BT, CT, TAT, WT, RT;
};

int main()
{
   int n;

   printf("Number of processes: ");
   scanf("%d", &n);

   struct processes p[n];

   for (int i = 0; i < n; i++)
   {
      p[i].ID = i;

      printf("Arrival time of P%d: ", p[i].ID);
      scanf("%d", &p[i].AT);

      printf("Burst time of P%d: ", p[i].ID);
      scanf("%d", &p[i].BT);
   }

   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n - i - 1; j++)
      {
         if (p[j].AT > p[j + 1].AT)
         {
            struct processes temp = p[j];
            p[j] = p[j + 1];
            p[j + 1] = temp;
         }
      }
   }

   int timepassed = 0;
   int sumTAT = 0, sumWT = 0, sumRT = 0;

   for (int i = 0; i < n; i++)
   {
      if (timepassed < p[i].AT)
      {
         timepassed = p[i].AT;
```

```
        }

        p[i].RT = timepassed - p[i].AT;

        p[i].CT = timepassed + p[i].BT;

        p[i].TAT = p[i].CT - p[i].AT;

        p[i].WT = p[i].TAT - p[i].BT;

        timepassed += p[i].BT;

        sumTAT += p[i].TAT;
        sumWT += p[i].WT;
        sumRT += p[i].RT;
    }

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");

    for (int i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].ID,
               p[i].AT,
               p[i].BT,
               p[i].CT,
               p[i].TAT,
               p[i].WT,
               p[i].RT);
    }

    printf("\nAverage TAT: %.2f\n", (float)sumTAT / n);
    printf("Average WT  : %.2f\n", (float)sumWT / n);
    printf("Average RT  : %.2f\n", (float)sumRT / n);

    return 0;
}
```

Output Screenshot –



```
Number of processes: 3
Arrival time of P0: 0
Burst time of P0: 4
Arrival time of P1: 3
Burst time of P1: 2
Arrival time of P2: 2
Burst time of P2: 5

Process AT      BT      CT      TAT     WT      RT
P0      0       4       4       4       0       0
P2      2       5       9       7       2       2
P1      3       2       11      8       6       6

Average TAT: 6.33
Average WT: 2.67
Average RT: 2.67

Process returned 0 (0x0)   execution time : 17.505 s
Press any key to continue.
```

Solution for SJF –

```
#include <stdio.h>

struct Process
{
   int id, at, bt, ct, tat, wt, rt;
};

int main()
{
   int n;

   printf("Enter no. of processes: ");
   scanf("%d", &n);

   struct Process p[n];

   for (int i = 0; i < n; i++)
   {
      p[i].id = i + 1;

      printf("Arrival Time of P%d: ", p[i].id);
      scanf("%d", &p[i].at);

      printf("Burst Time of P%d: ", p[i].id);
      scanf("%d", &p[i].bt);

      p[i].ct = 0;
      p[i].tat = 0;
      p[i].wt = 0;
      p[i].rt = 0;
   }

   int completed = 0;
   int time = 0;

   int sumTAT = 0;
   int sumWT = 0;
   int sumRT = 0;

   int isComplete[n];

   for (int i = 0; i < n; i++)
   {
      isComplete[i] = 0;
   }

   while (completed < n)
   {
      int idx = -1;

      for (int i = 0; i < n; i++)
      {
         if (p[i].at <= time && isComplete[i] == 0)
```

```
            {
                if (idx == -1 || p[i].bt < p[idx].bt)
                {
                    idx = i;
                }
            }
        }

        if (idx != -1)
        {
            p[idx].rt = time - p[idx].at;

            p[idx].ct = time + p[idx].bt;

            p[idx].tat = p[idx].ct - p[idx].at;

            p[idx].wt = p[idx].tat - p[idx].bt;

            sumTAT += p[idx].tat;
            sumWT += p[idx].wt;
            sumRT += p[idx].rt;

            time += p[idx].bt;

            isComplete[idx] = 1;
            completed++;
        }
        else
        {
            time++;
        }
    }

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");

    for (int i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].id,
               p[i].at,
               p[i].bt,
               p[i].ct,
               p[i].tat,
               p[i].wt,
               p[i].rt);
    }

    printf("\nAverage TAT: %.2f\n", (float)sumTAT / n);
    printf("Average WT  : %.2f\n", (float)sumWT / n);
    printf("Average RT  : %.2f\n", (float)sumRT / n);

    return 0;
}
```

Output Screenshot –

```
Enter no. of processes: 4
Arrival Time of P1: 0
Burst Time of P1: 7
Arrival Time of P2: 2
Burst Time of P2: 4
Arrival Time of P3: 4
Burst Time of P3: 1
Arrival Time of P4: 5
Burst Time of P4: 4

Process AT      BT      CT      TAT     WT
P1      0       7       7       7       0
P2      2       4       12      10      6
P3      4       1       8       4       3
P4      5       4       16      11      7
Average TAT: 8.00
Average WT: 4.00
```

Solution for SRTF –

```
#include <stdio.h>

struct Process
{
   int id, at, bt, ct, tat, wt, rt, rem;
};

int main()
{
   int n;

   printf("Enter no. of process: ");
   scanf("%d", &n);

   struct Process p[n];

   for (int i = 0; i < n; i++)
   {
      p[i].id = i + 1;

      printf("Arrival Time of P%d: ", p[i].id);
      scanf("%d", &p[i].at);

      printf("Burst Time of P%d: ", p[i].id);
      scanf("%d", &p[i].bt);

      p[i].rem = p[i].bt;
      p[i].rt = -1;
   }

   int completed = 0;
   int time = 0;

   int sumTAT = 0;
   int sumWT = 0;
   int sumRT = 0;

   while (completed != n)
   {
      int idx = -1;
      int minRem = 1000000;

      for (int i = 0; i < n; i++)
      {
         if (p[i].at <= time && p[i].rem > 0)
         {
            if (p[i].rem < minRem)
            {
               minRem = p[i].rem;
               idx = i;
            }
         }
      }
```

```
        if (idx != -1)
        {
            if (p[idx].rt == -1)
            {
                p[idx].rt = time - p[idx].at;
            }

            p[idx].rem--;

            time++;

            if (p[idx].rem == 0)
            {
                p[idx].ct = time;

                p[idx].tat = p[idx].ct - p[idx].at;

                p[idx].wt = p[idx].tat - p[idx].bt;

                sumTAT += p[idx].tat;
                sumWT += p[idx].wt;
                sumRT += p[idx].rt;

                completed++;
            }
        }
        else
        {
            time++;
        }
    }

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");

    for (int i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].id,
               p[i].at,
               p[i].bt,
               p[i].ct,
               p[i].tat,
               p[i].wt,
               p[i].rt);
    }

    printf("\nAvg TAT: %.2f\n", (float)sumTAT / n);
    printf("Avg WT  : %.2f\n", (float)sumWT / n);
    printf("Avg RT  : %.2f\n", (float)sumRT / n);

    return 0;
}
```

Output Screenshot –

```
Enter no. of process: 4
Arrival Time of P1: 0
Burst Time of P1: 8
Arrival Time of P2: 1
Burst Time of P2: 4
Arrival Time of P3: 2
Burst Time of P3: 2
Arrival Time of P4: 3
Burst Time of P4: 1

Process AT      BT      CT      TAT     WT      RT
P1      0       8       15      15      7       0
P2      1       4       8       7       3       0
P3      2       2       4       2       0       0
P4      3       1       5       2       1       1

Avg TAT: 6.50
Avg WT: 2.75
Avg RT: 0.25
```

PROGRAM 1b

Question : Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time for Priority and Round Robin (experimenting with different time quantums)

Solution for Priority Non Pre-emption –

```
#include <stdio.h>

struct Process
{
   int pid, at, bt, priority, ct, wt, tat, is_completed;
};

int main()
{
   int n;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   struct Process p[n];

   for (int i = 0; i < n; i++)
   {
      p[i].pid = i + 1;

      printf("Enter AT, BT and Priority for P%d: ", i + 1);
      scanf("%d %d %d",
            &p[i].at,
            &p[i].bt,
            &p[i].priority);

      p[i].is_completed = 0;
   }

   int completed = 0;
   int time = 0;

   int min_pri;
   int selected = -1;

   while (completed != n)
   {
      min_pri = 10000;
      selected = -1;

      for (int i = 0; i < n; i++)
      {
         if ((p[i].at <= time) &&
            (p[i].is_completed == 0) &&
            (p[i].priority < min_pri))
         {
            min_pri = p[i].priority;
```

```
            selected = i;
        }
    }

    if (selected == -1)
    {
        time++;
        continue;
    }

    time += p[selected].bt;

    p[selected].ct = time;

    p[selected].tat =
        p[selected].ct - p[selected].at;

    p[selected].wt =
        p[selected].tat - p[selected].bt;

    p[selected].is_completed = 1;

    completed++;
}

printf("\nPID\tAT\tBT\tPri\tCT\tWT\tTAT\n");

for (int i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
           p[i].pid,
           p[i].at,
           p[i].bt,
           p[i].priority,
           p[i].ct,
           p[i].wt,
           p[i].tat);
}

float avg_tat = 0;
float avg_wt = 0;

for (int i = 0; i < n; i++)
{
    avg_tat += p[i].tat;
    avg_wt += p[i].wt;
}

printf("Average Turn Around Time: %f\n",
       (avg_tat / n));

printf("Average Waiting Time: %f\n",
       (avg_wt / n));

return 0;
```

}

Output Screenshot –

```
Enter number of processes: 5
Enter AT, BT and Priority for P1: 0 3 5
Enter AT, BT and Priority for P2: 2 2 3
Enter AT, BT and Priority for P3: 3 5 2
Enter AT, BT and Priority for P4: 4 4 4
Enter AT, BT and Priority for P5: 6 1 1

PID     AT      BT      Pri     CT      WT      TAT
1       0       3       5       3       0       3
2       2       2       3       11      7       9
3       3       5       2       8       0       5
4       4       4       4       15      7       11
5       6       1       1       9       2       3
Average Turn Around Time: 6.200000
Average Waiting Time: 3.200000
```

Solution for Priority Pre-emption –

```
#include <stdio.h>

struct Process
{
   int pid, at, bt, rt, priority, ct, wt, tat;
};

int main()
{
   int n;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   struct Process p[n];

   for (int i = 0; i < n; i++)
   {
      p[i].pid = i + 1;

      printf("Enter AT, BT and Priority for P%d: ", i + 1);

      scanf("%d %d %d",
            &p[i].at,
            &p[i].bt,
            &p[i].priority);

      p[i].rt = p[i].bt;
   }

   int completed = 0;
   int time = 0;

   int min_pri;
   int shortest = -1;
   int check = 0;

   while (completed != n)
   {
      min_pri = 10000;
      shortest = -1;
      check = 0;

      for (int i = 0; i < n; i++)
      {
         if ((p[i].at <= time) &&
            (p[i].rt > 0) &&
            (p[i].priority < min_pri))
         {
            min_pri = p[i].priority;
            shortest = i;
            check = 1;
         }
```

```
    }

    if (check == 0)
    {
        time++;
        continue;
    }

    p[shortest].rt--;

    time++;

    if (p[shortest].rt == 0)
    {
        completed++;

        p[shortest].ct = time;

        p[shortest].tat =
            p[shortest].ct - p[shortest].at;

        p[shortest].wt =
            p[shortest].tat - p[shortest].bt;
    }
}

printf("\nPID\tAT\tBT\tPri\tCT\tWT\tTAT\n");

for (int i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
           p[i].pid,
           p[i].at,
           p[i].bt,
           p[i].priority,
           p[i].ct,
           p[i].wt,
           p[i].tat);
}

float avg_tat = 0;
float avg_wt = 0;

for (int i = 0; i < n; i++)
{
    avg_tat += p[i].tat;
    avg_wt += p[i].wt;
}

printf("Average Turn Around Time: %f\n",
       (avg_tat / n));

printf("Average Waiting Time: %f\n",
       (avg_wt / n));
```

```
    return 0;
}
```

Output Screenshot –

```
Enter number of processes: 7
Enter AT, BT and Priority for P1: 6 5 6
Enter AT, BT and Priority for P2: 4 1 5
Enter AT, BT and Priority for P3: 1 2 4
Enter AT, BT and Priority for P4: 3 4 4
Enter AT, BT and Priority for P5: 0 8 3
Enter AT, BT and Priority for P6: 5 6 2
Enter AT, BT and Priority for P7: 10 1 1

PID     AT      BT      Pri     CT      WT      TAT
1       6       5       6       27      16      21
2       4       1       5       22      17      18
3       1       2       4       17      14      16
4       3       4       4       21      14      18
5       0       8       3       15      7       15
6       5       6       2       12      1       7
7       10      1       1       11      0       1
Average Turn Around Time: 13.714286
Average Waiting Time: 9.857143
```

Solution for Round Robin –

```
#include <stdio.h>

struct Process
{
   int pid, at, bt, ct, tat, wt, rt;
};

int main()
{
   int n, tq;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter Time Quantum: ");
   scanf("%d", &tq);

   struct Process p[n];

   int rem_bt[n];

   int readyQ[100];

   int front = 0;
   int rear = 0;

   printf("Enter arrival times:\n");

   for (int i = 0; i < n; i++)
   {
      scanf("%d", &p[i].at);

      p[i].pid = i + 1;
   }

   printf("Enter burst times:\n");

   for (int i = 0; i < n; i++)
   {
      scanf("%d", &p[i].bt);

      rem_bt[i] = p[i].bt;

      p[i].rt = p[i].bt;

      p[i].ct = 0;
      p[i].tat = 0;
      p[i].wt = 0;
   }

   int time = 0;
   int completed = 0;
```

```
int visited[n];

for (int i = 0; i < n; i++)
{
   visited[i] = 0;
}

for (int i = 0; i < n; i++)
{
   if (p[i].at == 0)
   {
      readyQ[rear++] = i;
      visited[i] = 1;
   }
}

while (completed < n)
{
   if (front == rear)
   {
      time++;

      for (int i = 0; i < n; i++)
      {
         if (!visited[i] && p[i].at <= time)
         {
            readyQ[rear++] = i;
            visited[i] = 1;
         }
      }

      continue;
   }

   int idx = readyQ[front++];

   if (rem_bt[idx] > tq)
   {
      time += tq;

      rem_bt[idx] -= tq;

      p[idx].rt = rem_bt[idx];
   }
   else
   {
      time += rem_bt[idx];

      rem_bt[idx] = 0;

      p[idx].ct = time;

      p[idx].tat =
         p[idx].ct - p[idx].at;
```

```
            p[idx].wt =
                p[idx].tat - p[idx].bt;

            p[idx].rt = 0;

            completed++;
        }

        for (int i = 0; i < n; i++)
        {
            if (!visited[i] && p[i].at <= time)
            {
                readyQ[rear++] = i;
                visited[i] = 1;
            }
        }

        if (rem_bt[idx] > 0)
        {
            readyQ[rear++] = idx;
        }
    }

    printf("\nRRS scheduling:\n");

    printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
                p[i].pid,
                p[i].at,
                p[i].bt,
                p[i].ct,
                p[i].tat,
                p[i].wt,
                p[i].rt);
    }

    float avg_TAT = 0;
    float avg_WT = 0;

    for (int i = 0; i < n; i++)
    {
        avg_TAT += p[i].tat;
        avg_WT += p[i].wt;
    }

    printf("\nAverage turnaround time: %.2f ms\n",
            avg_TAT / n);

    printf("Average waiting time: %.2f ms\n",
            avg_WT / n);

    return 0;}
```

Output Screenshot –

```
Enter number of processes: 5
Enter Time Quantum: 2
Enter arrival times:
0 1 2 3 4
Enter burst times:
5 3 1 2 3

RRS scheduling:
PID     AT      BT      CT      TAT     WT      RT
1       0       5       13      13      8       0
2       1       3       12      11      8       0
3       2       1       5       3       2       0
4       3       2       9       6       4       0
5       4       3       14      10      7       0

Average turnaround time: 8.60 ms
Average waiting time: 5.80 ms
```

PROGRAM 2

Question : Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Solution for Multi-Level queue (1-RR,2-FCFS) –

```
#include <stdio.h>

struct Process
{
   int pid, at, bt, ct, tat, wt, queue;
};

int main()
{
   int n;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   struct Process p[n];

   int remaining[n];

   for (int i = 0; i < n; i++)
   {
      p[i].pid = i + 1;

      printf("Enter AT, BT, Queue (1/2) for P%d: ",
            p[i].pid);

      scanf("%d %d %d",
           &p[i].at,
           &p[i].bt,
           &p[i].queue);

      remaining[i] = p[i].bt;
   }

   int time = 0;
   int completed = 0;

   int tq = 2;

   while (completed < n)
   {
      int executed = 0;

      for (int i = 0; i < n; i++)
      {
         if (p[i].queue == 1 &&
            remaining[i] > 0 &&
```

```
            p[i].at <= time)
        {
            executed = 1;

            if (remaining[i] > tq)
            {
                time += tq;

                remaining[i] -= tq;
            }
            else
            {
                time += remaining[i];

                remaining[i] = 0;

                p[i].ct = time;

                p[i].tat =
                    p[i].ct - p[i].at;

                p[i].wt =
                    p[i].tat - p[i].bt;

                completed++;
            }
        }
    }

    if (!executed)
    {
        int idx = -1;

        for (int i = 0; i < n; i++)
        {
            if (p[i].queue == 2 &&
                remaining[i] > 0 &&
                p[i].at <= time)
            {
                idx = i;
                break;
            }
        }

        if (idx != -1)
        {
            time += remaining[idx];

            remaining[idx] = 0;

            p[idx].ct = time;

            p[idx].tat =
                p[idx].ct - p[idx].at;
```

```
            p[idx].wt =
               p[idx].tat - p[idx].bt;

            completed++;
         }
         else
         {
            time++;
         }
      }
   }

   printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tQueue\n");

   for (int i = 0; i < n; i++)
   {
      printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid,
            p[i].at,
            p[i].bt,
            p[i].ct,
            p[i].tat,
            p[i].wt,
            p[i].queue);
   }

   return 0;
}
```

Output Screenshot –

```
Enter number of processes: 5
Enter AT, BT, Queue (1/2) for P1: 0 4 2
Enter AT, BT, Queue (1/2) for P2: 3 4 1
Enter AT, BT, Queue (1/2) for P3: 4 1 1
Enter AT, BT, Queue (1/2) for P4: 2 8 2
Enter AT, BT, Queue (1/2) for P5: 1 5 2

PID     AT      BT      CT      TAT     WT      Queue
1       0       4       4       4       0       2
2       3       4       9       6       2       1
3       4       1       7       3       2       1
4       2       8       17      15      7       2
5       1       5       22      21      16      2
```

PROGRAM 3

QUESTION : Write a C program to simulate Real-Time CPU Scheduling algorithms:
Rate- Monotonic
Earliest-deadline First
Proportional scheduling

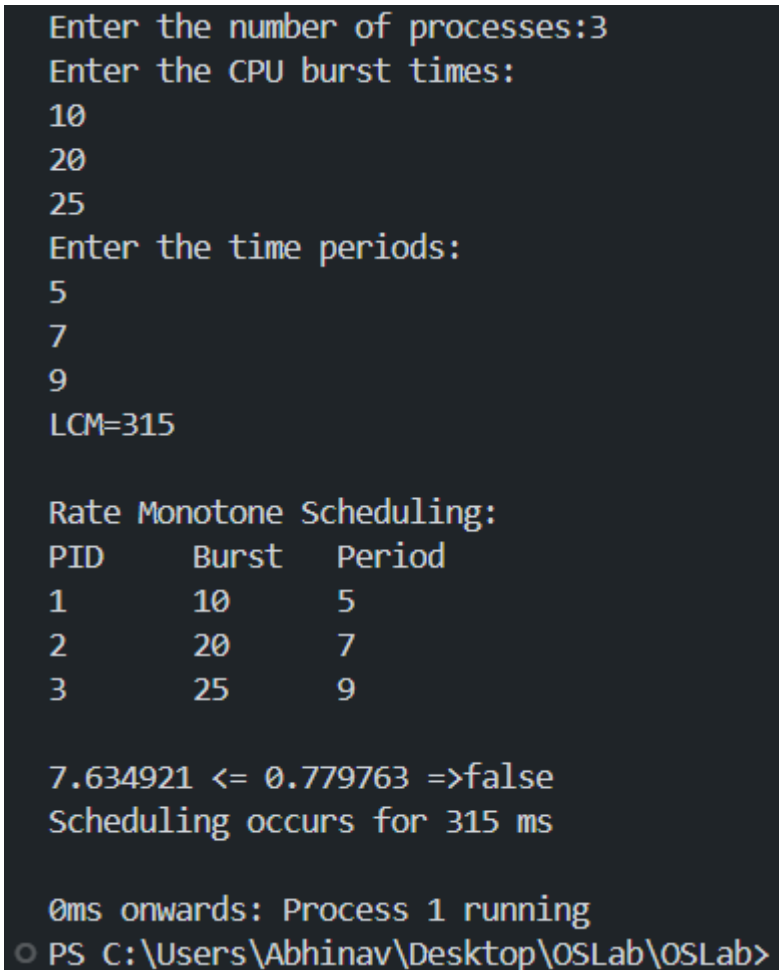
Solution for Rate-Monotonic –

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

struct Process
{
   int pid;
   int burst;
   int period;
   int remaining;
};

int gcd(int a, int b)
{
   if (b == 0)
   {
      return a;
   }

   return gcd(b, a % b);
}

int lcm(int a, int b)
{
   return (a * b) / gcd(a, b);
}

int main()
{
   int n;

   printf("Enter the number of processes: ");
   scanf("%d", &n);

   struct Process p[n];

   printf("Enter the CPU burst times:\n");

   for (int i = 0; i < n; i++)
   {
      scanf("%d", &p[i].burst);

      p[i].pid = i + 1;

      p[i].remaining = 0;
```

```
}

printf("Enter the time periods:\n");

int hyperPeriod = 1;

for (int i = 0; i < n; i++)
{
   scanf("%d", &p[i].period);

   hyperPeriod =
      lcm(hyperPeriod, p[i].period);
}

printf("LCM = %d\n\n", hyperPeriod);

printf("Rate Monotone Scheduling:\n");

printf("PID\tBurst\tPeriod\n");

for (int i = 0; i < n; i++)
{
   printf("%d\t%d\t%d\n",
         p[i].pid,
         p[i].burst,
         p[i].period);
}

double U = 0.0;

for (int i = 0; i < n; i++)
{
   U += (double)p[i].burst / p[i].period;
}

double bound =
   n * (pow(2.0, 1.0 / n) - 1);

printf("\n%.6f <= %.6f => %s\n",
      U,
      bound,
      (U <= bound ? "true" : "false"));

printf("Scheduling occurs for %d ms\n\n",
      hyperPeriod);

int prev_chosen = -2;

for (int t = 0; t < hyperPeriod; t++)
{
   for (int i = 0; i < n; i++)
   {
      if (t % p[i].period == 0)
      {
         p[i].remaining = p[i].burst;
```

```
            }
        }

        int chosen = -1;

        int min_period = 999999;

        for (int i = 0; i < n; i++)
        {
            if (p[i].remaining > 0 &&
                p[i].period < min_period)
            {
                min_period = p[i].period;

                chosen = i;
            }
        }

        if (chosen != prev_chosen)
        {
            if (chosen != -1)
            {
                printf("%dms onwards: Process %d running\n",
                       t,
                       p[chosen].pid);
            }
            else
            {
                printf("%dms onwards: CPU is idle\n",
                       t);
            }

            prev_chosen = chosen;
        }

        if (chosen != -1)
        {
            p[chosen].remaining--;
        }
    }

    return 0;
}
```

Output Screenshot –



Solution for Earliest-deadline first –

```
#include <stdio.h>

struct Task
{
   int pid;
   int burst;
   int deadline;
   int period;
   int rem;
   int abs_dl;
};

int gcd(int a, int b)
{
   if (b == 0)
   {
      return a;
   }

   return gcd(b, a % b);
}

int lcm(int a, int b)
{
   return (a * b) / gcd(a, b);
}

int main()
{
   int n;

   printf("Enter the number of processes: ");
   scanf("%d", &n);

   struct Task tks[n];

   int hyperperiod = 1;

   for (int i = 0; i < n; i++)
   {
      printf("Enter PID, Burst, Deadline, and Period for process %d: ",
            i + 1);

      scanf("%d %d %d %d",
           &tks[i].pid,
           &tks[i].burst,
           &tks[i].deadline,
           &tks[i].period);

      tks[i].rem = 0;

      tks[i].abs_dl = 0;
```

```
    hyperperiod =
        lcm(hyperperiod, tks[i].period);
}

printf("\nPID\tBurst\tDeadline\tPeriod\n");

for (int i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t\t%d\n",
           tks[i].pid,
           tks[i].burst,
           tks[i].deadline,
           tks[i].period);
}

printf("Scheduling occurs for %d ms\n\n",
       hyperperiod);

for (int time = 0; time < hyperperiod; time++)
{
    for (int i = 0; i < n; i++)
    {
        if (time % tks[i].period == 0)
        {
            tks[i].rem = tks[i].burst;

            tks[i].abs_dl =
                time + tks[i].deadline;
        }
    }

    int min_dl = 999999;

    int selected = -1;

    for (int i = 0; i < n; i++)
    {
        if (tks[i].rem > 0 &&
            tks[i].abs_dl < min_dl)
        {
            min_dl = tks[i].abs_dl;

            selected = i;
        }
    }

    if (selected != -1)
    {
        printf("%dms : Task %d is running.\n",
               time,
               tks[selected].pid);

        tks[selected].rem--;
    }
    else
```

```
        {
            printf("%dms : CPU is idle.\n",
                time);
        }
    }

    return 0;
}
```

Output Screenshot –

```
Enter the number of processes: 3
Enter PID, Burst, Deadline, and Period for process 1: 2 50 22 4
Enter PID, Burst, Deadline, and Period for process 2: 1 22 55 7
Enter PID, Burst, Deadline, and Period for process 3: 4 35 78 10

PID     Burst   Deadline        Period
2       50      22              4
1       22      55              7
4       35      78              10
Scheduling occurs for 140 ms
```

Solution for Proportional scheduling –

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct
{
   char name[5];

   int tickets;

} Process;

int main()
{
   int n;

   int total_tickets = 0;

   float total_T = 0.0;

   printf("Enter the number of Processes: ");

   scanf("%d", &n);

   Process p[n];

   srand(time(NULL));

   for (int i = 0; i < n; i++)
   {
      printf("\nProcess %d:\n", i + 1);

      sprintf(p[i].name, "P%d", i + 1);

      printf("Tickets: ");

      scanf("%d", &p[i].tickets);

      total_tickets += p[i].tickets;

      total_T += p[i].tickets;
   }

   printf("\n--- Proportional Share Scheduling ---\n");

   printf("Enter the Time Period for scheduling: ");

   int m;

   scanf("%d", &m);

   for (int i = 0; i < m; i++)
   {
```

```
        int winning_ticket =
            rand() % total_tickets + 1;

        int accumulated_tickets = 0;

        int winner_index;

        for (int j = 0; j < n; j++)
        {
            accumulated_tickets += p[j].tickets;

            if (winning_ticket <= accumulated_tickets)
            {
                winner_index = j;

                break;
            }
        }

        printf("Tickets picked: %d, Winner: %s\n",
               winning_ticket,
               p[winner_index].name);
    }

    for (int i = 0; i < n; i++)
    {
        printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n",
               p[i].name,
               ((p[i].tickets / total_T) * 100));
    }

    return 0;
}
```

Output Screenshot –

```
Enter the number of Processes: 3

Process 1:
Tickets: 35

Process 2:
Tickets: 30

Process 3:
Tickets: 25

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 10
Tickets picked: 44, Winner: P2
Tickets picked: 87, Winner: P3
Tickets picked: 69, Winner: P3
Tickets picked: 40, Winner: P2
Tickets picked: 29, Winner: P1
Tickets picked: 81, Winner: P3
Tickets picked: 57, Winner: P2
Tickets picked: 38, Winner: P2
Tickets picked: 46, Winner: P2
Tickets picked: 9, Winner: P1

The Process: P1 gets 38.89% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 27.78% of Processor Time.
```

PROGRAM 4 :

QUESTION :
4a. Write a C program to simulate producer-consumer problem using semaphores
4b. Write a C program to simulate the concept of Dining Philosophers problem

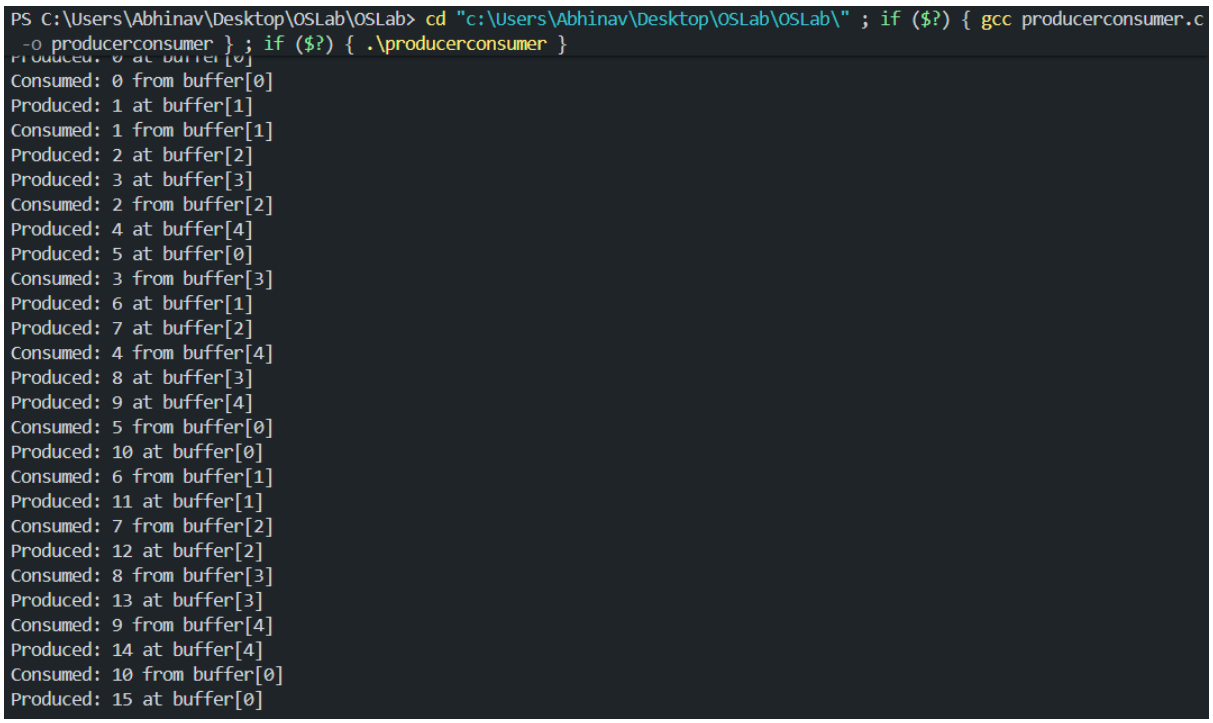
Solution for producer-consumer –

```
#include <stdio.h>
#include <windows.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

HANDLE empty;
HANDLE full;
HANDLE mutex;

DWORD WINAPI producer(LPVOID arg)
{
   int item = 0;

   while (1)
   {
      int i = item++;

      WaitForSingleObject(empty, INFINITE);

      WaitForSingleObject(mutex, INFINITE);

      buffer[in] = i;

      printf("Produced: %d at buffer[%d]\n",
            i,
            in);

      in = (in + 1) % BUFFER_SIZE;

      ReleaseMutex(mutex);

      ReleaseSemaphore(full, 1, NULL);

      Sleep(1000);
   }

   return 0;
}

DWORD WINAPI consumer(LPVOID arg)
{
```

```
    int item;

    while (1)
    {
        WaitForSingleObject(full, INFINITE);

        WaitForSingleObject(mutex, INFINITE);

        item = buffer[out];

        printf("Consumed: %d from buffer[%d]\n",
               item,
               out);

        out = (out + 1) % BUFFER_SIZE;

        ReleaseMutex(mutex);

        ReleaseSemaphore(empty, 1, NULL);

        Sleep(2000);
    }

    return 0;
}

int main()
{
    DWORD prodThreadId;
    DWORD consThreadId;

    empty =
        CreateSemaphore(NULL,
                        BUFFER_SIZE,
                        BUFFER_SIZE,
                        NULL);

    full =
        CreateSemaphore(NULL,
                        0,
                        BUFFER_SIZE,
                        NULL);

    mutex =
        CreateMutex(NULL,
                    FALSE,
                    NULL);

    HANDLE prodThread =
        CreateThread(NULL,
                     0,
                     producer,
                     NULL,
                     0,
                     &prodThreadId);
```

```
    HANDLE consThread =
        CreateThread(NULL,
                     0,
                     consumer,
                     NULL,
                     0,
                     &consThreadId);

    WaitForSingleObject(prodThread, INFINITE);

    WaitForSingleObject(consThread, INFINITE);

    CloseHandle(empty);

    CloseHandle(full);

    CloseHandle(mutex);

    CloseHandle(prodThread);

    CloseHandle(consThread);

    return 0;
}
```

Output Screenshot –



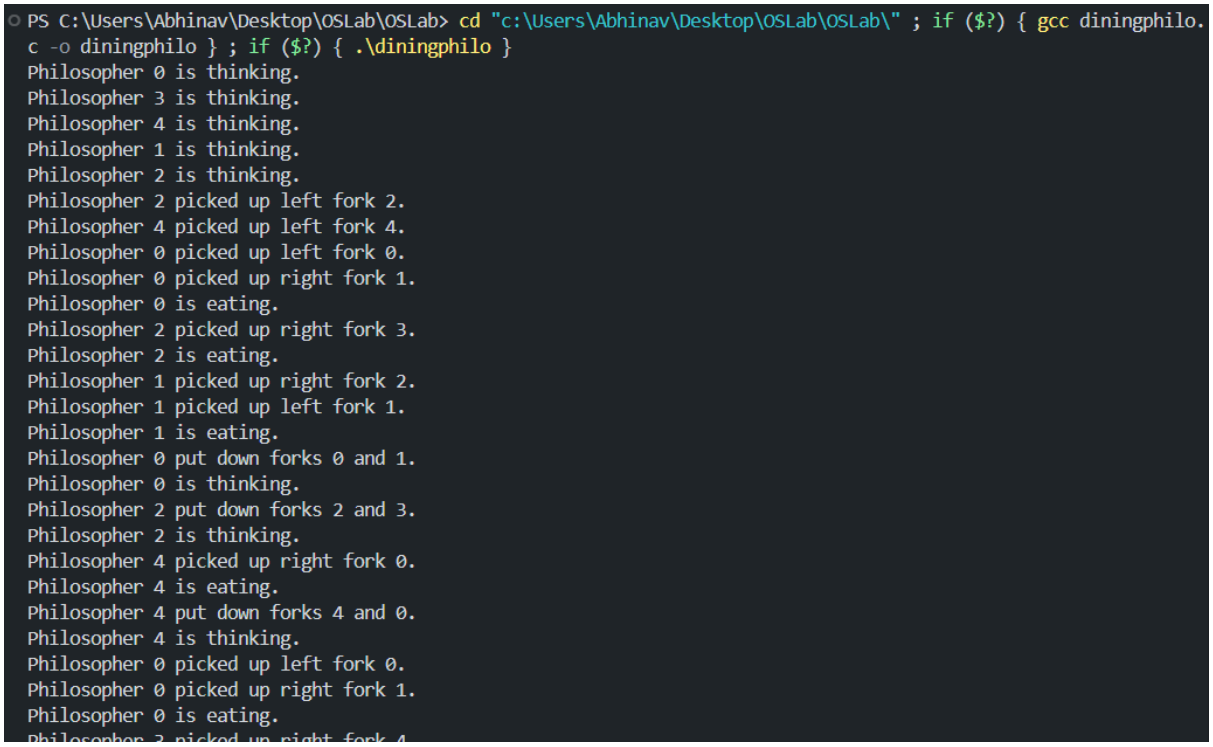
Solution for Dining Philosophers –

```
#include <stdio.h>
#include <windows.h>

#define N 5

HANDLE forks[N];

HANDLE philosophers[N];

DWORD WINAPI philosopher(LPVOID num)
{
   int id = *(int *)num;

   int left = id;

   int right = (id + 1) % N;

   while (1)
   {
      printf("Philosopher %d is thinking.\n",
            id);

      Sleep(1000);

      if (id % 2 == 0)
      {
         WaitForSingleObject(forks[left],
                        INFINITE);

         printf("Philosopher %d picked up left fork %d.\n",
               id,
               left);

         WaitForSingleObject(forks[right],
                        INFINITE);

         printf("Philosopher %d picked up right fork %d.\n",
               id,
               right);
      }
      else
      {
         WaitForSingleObject(forks[right],
                        INFINITE);

         printf("Philosopher %d picked up right fork %d.\n",
               id,
               right);

         WaitForSingleObject(forks[left],
                        INFINITE);

         printf("Philosopher %d picked up left fork %d.\n",
```

```
            id,
            left);
    }

    printf("Philosopher %d is eating.\n",
        id);

    Sleep(2000);

    ReleaseMutex(forks[left]);

    ReleaseMutex(forks[right]);

    printf("Philosopher %d put down forks %d and %d.\n",
        id,
        left,
        right);
  }

  return 0;
}

int main()
{
  int ids[N];

  for (int i = 0; i < N; i++)
  {
    forks[i] =
      CreateMutex(NULL,
            FALSE,
            NULL);

    ids[i] = i;
  }

  for (int i = 0; i < N; i++)
  {
    philosophers[i] =
      CreateThread(NULL,
            0,
            philosopher,
            &ids[i],
            0,
            NULL);
  }

  WaitForMultipleObjects(N,
            philosophers,
            TRUE,
            INFINITE);

  for (int i = 0; i < N; i++)
  {
    CloseHandle(forks[i]);
```

```
        CloseHandle(philosophers[i]);
    }

    return 0;
}
```

Output Screenshot –



PROGRAM 5

QUESTION : Write a C program to simulate Bankers algorithm and deadlock avoidance

Solution for Bankers algorithm –

```
#include <stdio.h>
#include <stdbool.h>

#define N 5  // Number of processes
#define M 3  // Number of resources

void calculateNeed(int need[N][M], int max[N][M], int alloc[N][M]) {
   for (int i = 0; i < N; i++) {
      for (int j = 0; j < M; j++) {
         need[i][j] = max[i][j] - alloc[i][j];
      }
   }
}

bool isSafe(int processes[], int avail[], int max[][M], int alloc[][M]) {
   int need[N][M];
   calculateNeed(need, max, alloc);

   bool finish[N] = {false};
   int safeSeq[N];
   int work[M];

   for (int i = 0; i < M; i++)
      work[i] = avail[i];

   int count = 0;
   while (count < N) {
      bool found = false;
      for (int p = 0; p < N; p++) {
         if (finish[p] == false) {
            int j;
            for (j = 0; j < M; j++)
               if (need[p][j] > work[j])
                  break;

            if (j == M) {
               for (int k = 0; k < M; k++)
                  work[k] += alloc[p][k];
               safeSeq[count++] = p;
               finish[p] = true;
               found = true;
            }
         }
      }

      if (found == false) {
         printf("The system is not in a safe state\n");
         return false;
```

```
        }
    }

    printf("The system is in a safe state.\n");
    printf("Safe sequence is: ");
    for (int i = 0; i < N; i++)
        printf("P%d ", safeSeq[i]);
    printf("\n");
    return true;
}

int main() {
    int processes[] = {0, 1, 2, 3, 4};
    int avail[] = {3, 3, 2};

    int max[N][M] = {
        {7, 5, 3},
        {3, 2, 2},
        {9, 0, 2},
        {2, 2, 2},
        {4, 3, 3}
    };

    int alloc[N][M] = {
        {0, 1, 0},
        {2, 0, 0},
        {3, 0, 2},
        {2, 1, 1},
        {0, 0, 2}
    };

    printf("Banker's Algorithm - Deadlock Avoidance\n");
    printf("=======================================\n");
    isSafe(processes, avail, max, alloc);

    return 0;
}
```

Output Screenshot –

```
PS C:\Users\Abhinav\Desktop\OSLab> cd "c:\Users\Abhinav\Desktop\OSLab\" ; if ($?) { gcc bankers.c -o bankers } ; if ($?) { .\bankers }
Banker's Algorithm - Deadlock Avoidance
=======================================
The system is in a safe state.
Safe sequence is: P1 P3 P4 P0 P2
PS C:\Users\Abhinav\Desktop\OSLab>
```

Solution for deadlock avoidance –

```
#include <stdio.h>
#include <stdbool.h>

#define N 5  // Number of processes
#define M 3  // Number of resources

void calculateNeed(int need[N][M], int max[N][M], int alloc[N][M]) {
   for (int i = 0; i < N; i++) {
      for (int j = 0; j < M; j++) {
         need[i][j] = max[i][j] - alloc[i][j];
      }
   }
}

bool isSafe(int processes[], int avail[], int max[][M], int alloc[][M]) {
   int need[N][M];
   calculateNeed(need, max, alloc);

   bool finish[N] = {false};
   int safeSeq[N];
   int work[M];

   for (int i = 0; i < M; i++)
      work[i] = avail[i];

   int count = 0;
   while (count < N) {
      bool found = false;
      for (int p = 0; p < N; p++) {
         if (finish[p] == false) {
            int j;
            for (j = 0; j < M; j++)
               if (need[p][j] > work[j])
                  break;

            if (j == M) {
               for (int k = 0; k < M; k++)
                  work[k] += alloc[p][k];
               safeSeq[count++] = p;
               finish[p] = true;
               found = true;
            }
         }
      }

      if (found == false) {
         printf("The system is not in a safe state\n");
         return false;
      }
   }

   printf("The system is in a safe state.\n");
   printf("Safe sequence is: ");
```

```
    for (int i = 0; i < N; i++)
        printf("P%d ", safeSeq[i]);
    printf("\n");
    return true;
}

int main() {
    int processes[] = {0, 1, 2, 3, 4};
    int avail[] = {3, 3, 2};

    int max[N][M] = {
        {7, 5, 3},
        {3, 2, 2},
        {9, 0, 2},
        {2, 2, 2},
        {4, 3, 3}
    };

    int alloc[N][M] = {
        {0, 1, 0},
        {2, 0, 0},
        {3, 0, 2},
        {2, 1, 1},
        {0, 0, 2}
    };

    printf("Banker's Algorithm - Deadlock Avoidance\n");
    printf("=======================================\n");
    isSafe(processes, avail, max, alloc);

    return 0;
}
```

Solution for Deadlock Detection –

```
#include <stdio.h>
#include <stdbool.h>

#define N 5
#define M 3

void deadlockDetection(int avail[],
                       int alloc[][M],
                       int request[][M])
{
    int work[M];

    bool finish[N];

    for (int i = 0; i < M; i++)
    {
        work[i] = avail[i];
    }

    for (int i = 0; i < N; i++)
```

```
{
    bool hasRequest = false;

    for (int j = 0; j < M; j++)
    {
        if (request[i][j] != 0)
        {
            hasRequest = true;

            break;
        }
    }

    finish[i] = !hasRequest;
}

int count = 0;

while (count < N)
{
    bool found = false;

    for (int i = 0; i < N; i++)
    {
        if (finish[i] == false)
        {
            bool canAllocate = true;

            for (int j = 0; j < M; j++)
            {
                if (request[i][j] > work[j])
                {
                    canAllocate = false;

                    break;
                }
            }

            if (canAllocate)
            {
                for (int j = 0; j < M; j++)
                {
                    work[j] += alloc[i][j];
                }

                finish[i] = true;

                count++;

                found = true;
            }
        }
    }

    if (!found)
```

```
        {
            break;
        }
    }

    bool deadlock = false;

    printf("\nDeadlock Detection Results:\n");

    printf("============================\n");

    for (int i = 0; i < N; i++)
    {
        if (finish[i] == false)
        {
            printf("Process P%d is deadlocked\n",
                   i);

            deadlock = true;
        }
        else
        {
            printf("Process P%d is not deadlocked\n",
                   i);
        }
    }

    if (deadlock)
    {
        printf("\nSystem is in DEADLOCKED state!\n");
    }
    else
    {
        printf("\nSystem is NOT deadlocked.\n");
    }
}

int main()
{
    int avail[] =
    {
        0, 0, 0
    };

    int alloc[N][M] =
    {
        {0, 1, 0},
        {2, 0, 0},
        {3, 0, 3},
        {2, 1, 1},
        {0, 0, 2}
    };

    int request[N][M] =
    {
```

```
        {0, 0, 0},
        {2, 0, 2},
        {0, 0, 0},
        {1, 0, 0},
        {0, 0, 2}
    };

    printf("Deadlock Detection - Multiple Instance Resources\n");

    printf("====================================================\n");

    printf("Available resources: ");

    for (int i = 0; i < M; i++)
    {
        printf("%d ", avail[i]);
    }

    printf("\n");

    deadlockDetection(avail,
                      alloc,
                      request);

    return 0;
}
```

Output Screenshot –

PROGRAM 6 :

QUESTION : Write a C program to simulate Memory Allocation (First Fit, Best Fit, Worst Fit)

Solution for Memory allocation –

```
#include <stdio.h>
#include <stdlib.h>

void firstFit(int blockSize[],
              int m,
              int processSize[],
              int n)
{
    int allocation[n];

    for (int i = 0; i < n; i++)
    {
        allocation[i] = -1;
    }

    printf("\n=== FIRST FIT ===\n");

    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                allocation[i] = j;

                blockSize[j] -= processSize[i];

                break;
            }
        }

        if (allocation[i] != -1)
        {
            printf("P%d\t\t%d\t\t%d\n",
                   i,
                   processSize[i],
                   allocation[i] + 1);
        }
        else
        {
            printf("P%d\t\t%d\t\tNot Allocated\n",
                   i,
                   processSize[i]);
        }
    }
}
```

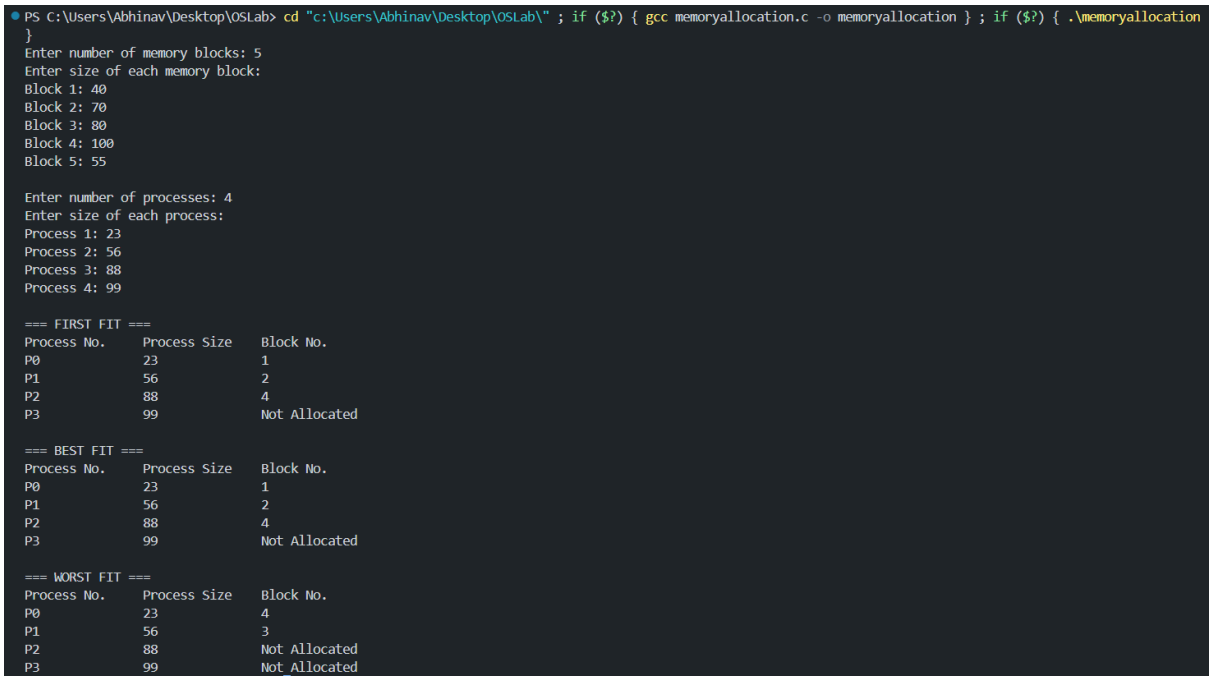
```
void bestFit(int blockSize[],
             int m,
             int processSize[],
             int n)
{
   int allocation[n];

   for (int i = 0; i < n; i++)
   {
      allocation[i] = -1;
   }

   printf("\n=== BEST FIT ===\n");

   printf("Process No.\tProcess Size\tBlock No.\n");

   for (int i = 0; i < n; i++)
   {
      int bestIdx = -1;

      for (int j = 0; j < m; j++)
      {
         if (blockSize[j] >= processSize[i])
         {
            if (bestIdx == -1 ||
                blockSize[j] < blockSize[bestIdx])
            {
                bestIdx = j;
            }
         }
      }

      if (bestIdx != -1)
      {
         allocation[i] = bestIdx;

         blockSize[bestIdx] -= processSize[i];

         printf("P%d\t\t%d\t\t%d\n",
                i,
                processSize[i],
                bestIdx + 1);
      }
      else
      {
         printf("P%d\t\t%d\t\tNot Allocated\n",
                i,
                processSize[i]);
      }
   }
}

void worstFit(int blockSize[],
              int m,
```

```
            int processSize[],
            int n)
{
    int allocation[n];

    for (int i = 0; i < n; i++)
    {
        allocation[i] = -1;
    }

    printf("\n=== WORST FIT ===\n");

    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < n; i++)
    {
        int worstIdx = -1;

        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                if (worstIdx == -1 ||
                    blockSize[j] > blockSize[worstIdx])
                {
                    worstIdx = j;
                }
            }
        }

        if (worstIdx != -1)
        {
            allocation[i] = worstIdx;

            blockSize[worstIdx] -= processSize[i];

            printf("P%d\t\t%d\t\t%d\n",
                   i,
                   processSize[i],
                   worstIdx + 1);
        }
        else
        {
            printf("P%d\t\t%d\t\tNot Allocated\n",
                   i,
                   processSize[i]);
        }
    }
}

int main()
{
    int m, n;

    printf("Enter number of memory blocks: ");
```

```
scanf("%d", &m);

int *blockSize =
   (int *)malloc(m * sizeof(int));

int *originalBlockSize =
   (int *)malloc(m * sizeof(int));

printf("Enter size of each memory block:\n");

for (int i = 0; i < m; i++)
{
   printf("Block %d: ", i + 1);

   scanf("%d", &blockSize[i]);

   originalBlockSize[i] = blockSize[i];
}

printf("\nEnter number of processes: ");

scanf("%d", &n);

int *processSize =
   (int *)malloc(n * sizeof(int));

printf("Enter size of each process:\n");

for (int i = 0; i < n; i++)
{
   printf("Process %d: ", i + 1);

   scanf("%d", &processSize[i]);
}

for (int i = 0; i < m; i++)
{
   blockSize[i] = originalBlockSize[i];
}

firstFit(blockSize,
        m,
        processSize,
        n);

for (int i = 0; i < m; i++)
{
   blockSize[i] = originalBlockSize[i];
}

bestFit(blockSize,
       m,
       processSize,
       n);
```

```
    for (int i = 0; i < m; i++)
    {
        blockSize[i] = originalBlockSize[i];
    }

    worstFit(blockSize,
             m,
             processSize,
             n);

    free(blockSize);

    free(originalBlockSize);

    free(processSize);

    return 0;
}
```

Output Screenshot –



```
PS C:\Users\Abhinav\Desktop\OSLab> cd "c:\Users\Abhinav\Desktop\OSLab\" ; if ($?) { gcc memoryallocation.c -o memoryallocation } ; if ($?) { .\memoryallocation
}
Enter number of memory blocks: 5
Enter size of each memory block:
Block 1: 40
Block 2: 70
Block 3: 80
Block 4: 100
Block 5: 55

Enter number of processes: 4
Enter size of each process:
Process 1: 23
Process 2: 56
Process 3: 88
Process 4: 99

=== FIRST FIT ===
Process No.     Process Size    Block No.
P0              23              1
P1              56              2
P2              88              4
P3              99              Not Allocated

=== BEST FIT ===
Process No.     Process Size    Block No.
P0              23              1
P1              56              2
P2              88              4
P3              99              Not Allocated

=== WORST FIT ===
Process No.     Process Size    Block No.
P0              23              4
P1              56              3
P2              88              Not Allocated
P3              99              Not Allocated
```

PROGRAM 7 :

QUESTION : Write a C program to simulate Page Replacement

Solution for Page Replacement –

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

int frames, pages;

int pageRef[100];

int frame[10];

int hit = 0;
int miss = 0;

void fifo()
{
   int stack[10] = {0};

   int top = 0;

   int pageNo[10] = {0};

   int flag;

   int count = 0;

   printf("\n=== FIFO PAGE REPLACEMENT ===\n");

   printf("Page\tFrame Array\n");

   for (int i = 0; i < pages; i++)
   {
      flag = 0;

      for (int j = 0; j < frames; j++)
      {
         if (frame[j] == pageRef[i])
         {
            flag = 1;

            hit++;
```

```
            break;
        }
    }

    if (flag == 0)
    {
        if (count < frames)
        {
            frame[count] = pageRef[i];

            pageNo[top] = pageRef[i];

            stack[top++] = count;

            count++;

            miss++;
        }
        else
        {
            int pos = stack[0];

            for (int j = 0; j < top - 1; j++)
            {
                stack[j] = stack[j + 1];

                pageNo[j] = pageNo[j + 1];
            }

            top--;

            frame[pos] = pageRef[i];

            pageNo[top] = pageRef[i];

            stack[top++] = pos;

            miss++;
        }
    }

    printf("%d\t", pageRef[i]);

    for (int j = 0; j < frames; j++)
    {
        printf("%d ", frame[j]);
    }
```

```
        printf("\n");
    }
}

void lru()
{
    int frameAge[10];

    printf("\n=== LRU PAGE REPLACEMENT ===\n");

    printf("Page\tFrame Array\n");

    for (int i = 0; i < frames; i++)
    {
        frame[i] = -1;
    }

    for (int i = 0; i < pages; i++)
    {
        int found = 0;

        for (int j = 0; j < frames; j++)
        {
            if (frame[j] == pageRef[i])
            {
                found = 1;

                frameAge[j] = i;

                hit++;

                break;
            }
        }

        if (!found)
        {
            int minAge = 999999;

            int pos = 0;

            int emptyPos = -1;

            for (int j = 0; j < frames; j++)
            {
                if (frame[j] == -1)
```

```
            {
                emptyPos = j;

                break;
            }

            if (frameAge[j] < minAge)
            {
                minAge = frameAge[j];

                pos = j;
            }
        }

        if (emptyPos != -1)
        {
            frame[emptyPos] = pageRef[i];

            frameAge[emptyPos] = i;
        }
        else
        {
            frame[pos] = pageRef[i];

            frameAge[pos] = i;
        }

        miss++;
    }

    printf("%d\t", pageRef[i]);

    for (int j = 0; j < frames; j++)
    {
        printf("%d ", frame[j]);
    }

    printf("\n");
  }
}

void optimal()
{
  printf("\n=== OPTIMAL PAGE REPLACEMENT ===\n");

  printf("Page\tFrame Array\n");
```

```
for (int i = 0; i < frames; i++)
{
   frame[i] = -1;
}

for (int i = 0; i < pages; i++)
{
   int found = 0;

   for (int j = 0; j < frames; j++)
   {
      if (frame[j] == pageRef[i])
      {
         found = 1;

         hit++;

         break;
      }
   }

   if (!found)
   {
      int farthest = i;

      int pos = 0;

      int emptyPos = -1;

      for (int j = 0; j < frames; j++)
      {
         if (frame[j] == -1)
         {
            emptyPos = j;

            break;
         }

         int k;

         for (k = i + 1; k < pages; k++)
         {
            if (frame[j] == pageRef[k])
            {
               break;
            }
         }
```

```
            if (k == pages && farthest == i)
            {
                farthest = k;

                pos = j;
            }
            else if (k > farthest)
            {
                farthest = k;

                pos = j;
            }
        }

        if (emptyPos != -1)
        {
            frame[emptyPos] = pageRef[i];
        }
        else
        {
            frame[pos] = pageRef[i];
        }

        miss++;
    }

    printf("%d\t", pageRef[i]);

    for (int j = 0; j < frames; j++)
    {
        printf("%d ", frame[j]);
    }

    printf("\n");
  }
}

int main()
{
  printf("Enter number of frames: ");

  scanf("%d", &frames);

  printf("Enter number of page references: ");

  scanf("%d", &pages);
```

```
    printf("Enter page reference string:\n");

    for (int i = 0; i < pages; i++)
    {
        scanf("%d", &pageRef[i]);
    }

    fifo();

    hit = miss = 0;

    for (int i = 0; i < frames; i++)
    {
        frame[i] = -1;
    }

    lru();

    hit = miss = 0;

    for (int i = 0; i < frames; i++)
    {
        frame[i] = -1;
    }

    optimal();

    printf("\n=== SUMMARY ===\n");

    printf("Total Hits: %d\n", hit);

    printf("Total Misses: %d\n", miss);

    printf("Hit Rate: %.2f%%\n",
           (float)hit / (hit + miss) * 100);

    printf("Miss Rate: %.2f%%\n",
           (float)miss / (hit + miss) * 100);

    return 0;
}
```

Output Screenshots –